

What is IAAS & IAC?

IAAS:

- IAAS is Infrastructure as a service. Where we can get Infrastructure on-demand. These type of services are provided by [AWS, GCP, Azure & Digital Ocean]

IAC:

- IAC is Infrastructure as code. Through code we can pass set of instructions which is needed to provision and configure our infrastructure.

Terraform

- Using Terraform we can write our own code to setup your infrastructure.
- Hence basically it's fit into IAC.

Introduction to Terraform:

- Terraform is an open-source Infrastructure as a code software tool developed by Hashicorp.
- It helps users to define and provision datacentre infrastructure using language known as Hashicorp Configuration Language (HCL)

Why Terraform?

Automation

Helps you to automate your infrastructure rather than doing things manually.

Flexible

Easy to manage multiple cloud providers such as [AWS, AZURE, GCP, Digital Ocean] and private infrastructure such as

Open source

Free to deploy and use.

Managing State

Helps to keep your infrastructure in certain state.

Auditable

- By seeing the code, you can understand what our infrastructure is made of.
- Also you can keep your infrastructure change history in version control system like Git.

Easy to use

- Easy to install and use.
- Codes written are human readable.

Terraform Vs other tools

Chef, Puppet & Ansible	Cloud Formation, Heat	Python Boto	Custom solutions
Configuration management tools are used to install and manage software on the machine that already exist	Cloud formation & heat are native solutions provided by their own cloud providers which helps to code the infrastructure into a configuration file.	Libraries like Boto are used to provide native access to cloud providers and services by using their API's.	Many organizations starts to setup their own infrastructure solutions either through simple scripts or web-based interfaces.
Terraform is predominantly used to automate the infrastructure itself.	Terraform does the same, you can code the infrastructure setup into configuration file and implement.	Terraform is very flexible and uses plugin-based model to support providers.	Difficulty here is when infrastructure grows it might become tedious to manage your custom solutions.
Terraform is not a Configuration management tool, and it allows and existing tools to focus on their strengths.	Major difference here is Terraform is cloud-agnostic. It can work with multiple public & private providers.	Major difference here is libraries might give only low-level programmatic access through APIs but Terraform gives high-level access.	Terraform is designed to tackle these kind of challenges. It provides simple solution to manage your Infrastructure.

Common use cases of Terraform:

Multi-Tier Applications:

The very common pattern is 2-tier architecture [Pool of Web servers along with DB Servers]. Additional tiers may get added based on requirement. Terraform is very useful to handle in setting up this kind of Infrastructures.

Disposable Environment:

It's very easy to setup a test environment when and where required and it's not going to be complex anymore.

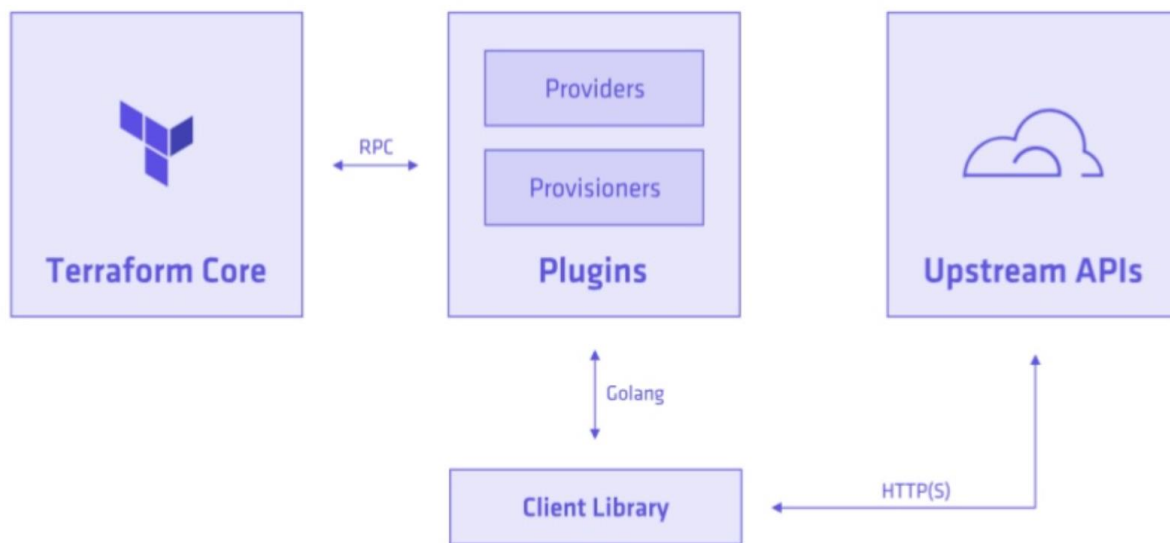
Multi-Cloud Deployment:

It's often attractive to spread infrastructure across multiple clouds to increase FT. This is very much possible through Terraforms.

How Terraform works?

- Terraform is built on a plugin-based architecture.
- This helps developers to extend Terraform by writing new plugins or compiling modified versions of existing plugins.
- Terraform is logically split into two main parts [Terraform Core & Terraform Plugins]
- **Terraform Core :**
 - Terraform Core uses remote procedure calls (RPC) to communicate with Terraform Plugins.
 - Terraform Core is written in Go programming language. The code is open source and you can find it GitHub.
- **Terraform Plugins :**
 - Terraform Plugins are also written in Go programming language.
 - Plugins are used in configuration files.
 - Plugins consist of 2 things [Providers & Provisioners]

Architecture of Terraform



Infrastructure Prerequisite

- We just need one AWS account for practise.
- One Linux machines from where you can keep your codes and apply.

Open AWS account

- As mentioned earlier we are going to see terraform with AWS in coming sessions. Hence to practice on your own you might need an AWS account.
- Most of you all know, AWS provides certain service for free on first year of your subscription so anyone can open new account with AWS and start exploring it. [AWS Link](#)
- With this free account you can start spinning up some basic configuration VMs [Which is called free tire EC2 instances] in AWS.
- Do note: Things might cost you when you are intend to use AWS free platform.
 - Using any others services such as S3, RDS etc.
 - Spinning up high resource machines apart from free tire.

Create IAM admin user

- Navigate to IAM in AWS console.
- Create an user called terraform [We are going to perform all automation activities through this IAM account from Terraform]
 - Select create individual IAM users
 - username = terraform
 - Enable programmatic access [Because we are going to access this user only through program not for login]
 - Create new group = terraform-administrator
 - Provide admin access to created group.
 - Add user terraform to group terraform-administrator.
 - Once user is created – You will get [Access key ID & Secret key]
 - You can also download the keys as CSV file.

Account final setup

- User & Group creation :

User details

User name	terraform
AWS access type	Programmatic access - with an access key
Permissions boundary	Permissions boundary is not set

Permissions summary

The user shown above will be added to the following groups.

Type	Name
Group	terraform-administrator

- Secret access Keys:



Success

You successfully created the users shown below. You can view and download user security credentials. You can also email users instructions for signing in to the AWS Management Console. This is the last time these credentials will be available to download. However, you can create new credentials at any time.

Users with AWS Management Console access can sign-in at: <https://119476179061.signin.aws.amazon.com/console>

Download .csv

	User	Access key ID	Secret access key
▶	✓ terraform		

- Excel file :

User name	Password	Access key ID	Secret access key	Console login link
terraform		AKIAI4478V90007		https://119476179001.signin.aws.amazon.com/console